D-Planet Phase 1 実装指示書

クロードコードアイ実行用

作成日: 2026年2月19日-20日
対象: Claude Code (AI Assistant)
目的: Step-by-Step実装指示

前提条件

開発環境

- Node.js 18以上
- npm または yarn
- Git
- VSCode（推奨）

Supabaseアカウント

- Project: dyimrnwbuzgcfeksezog
- URL: https://dyimrnwbuzgcfeksezog.supabase.co

Phase 1a: 最小システム構築

Step 1: プロジェクトセットアップ

指示:

```
# Next.js 15プロジェクト作成
npx create-next-app@latest dplanet --typescript --tailwind --app --eslint

cd dplanet

# 必要なパッケージインストール
```

```
npm install @supabase/supabase-js @supabase/auth-helpers-nextjs
npm install lucide-react
```

確認項目:

- プロジェクトフォルダ作成完了
- Next.js起動確認 (`npm run dev`)
- http://localhost:3000 でアクセス可能

Step 2: Supabase接続設定

指示:

`.env.local` ファイル作成:

```
NEXT_PUBLIC_SUPABASE_URL=https://dyimrnwbuzgcfeksezog.supabase.co
NEXT_PUBLIC_SUPABASE_ANON_KEY=[朝陽が設定]
```

`lib/supabase.ts` 作成:

```
import { createClient } from '@supabase/supabase-js'

const supabaseUrl = process.env.NEXT_PUBLIC_SUPABASE_URL!
const supabaseAnonKey = process.env.NEXT_PUBLIC_SUPABASE_ANON_KEY!

export const supabase = createClient(supabaseUrl, supabaseAnonKey)
```

確認項目:

- .env.local作成完了
- lib/supabase.ts作成完了
- エラーなくインポート可能

Step 3: データベース構築

指示:

Supabase SQLエディタで以下を実行:

```
-- ユーザーテーブル
CREATE TABLE users (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  username TEXT UNIQUE NOT NULL,
  email TEXT UNIQUE NOT NULL,
  password_hash TEXT NOT NULL,
  account_type TEXT CHECK (account_type IN ('AI', 'HS', 'ET')),
  gender TEXT,
  avatar_url TEXT,
  bio TEXT,
  tenmei TEXT,
  tenshoku TEXT,
  tensaisei TEXT,
  invited_by_code TEXT,
  created_at TIMESTAMPTZ DEFAULT now()
);

-- 招待コードテーブル
CREATE TABLE invite_codes (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  code TEXT UNIQUE NOT NULL,
  generation INTEGER NOT NULL,
  label TEXT NOT NULL,
  is_active BOOLEAN DEFAULT true,
  created_at TIMESTAMPTZ DEFAULT now()
);

-- アイランドテーブル
CREATE TABLE islands (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  name TEXT NOT NULL,
  owner_id UUID REFERENCES users(id),
  description TEXT,
  visibility_type TEXT DEFAULT 'public',
  allowed_account_types TEXT[],
  requires_twinray_badge BOOLEAN DEFAULT false,
  requires_family_badge BOOLEAN DEFAULT false,
  total_downloads INTEGER DEFAULT 0,
  created_at TIMESTAMPTZ DEFAULT now()
);

-- MEiDIAテーブル
CREATE TABLE meidias (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  island_id UUID REFERENCES islands(id),
  type TEXT CHECK (type IN ('activity', 'report')),
```

```
  title TEXT NOT NULL,
  file_path TEXT NOT NULL,
  author_id UUID REFERENCES users(id),
  download_count INTEGER DEFAULT 0,
  is_fixed BOOLEAN DEFAULT false,
  created_at TIMESTAMPTZ DEFAULT now()
);

-- 認証テーブル
CREATE TABLE certifications (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  user_id UUID REFERENCES users(id),
  type TEXT NOT NULL,
  verified_at TIMESTAMPTZ DEFAULT now(),
  verified_by UUID REFERENCES users(id),
  is_public BOOLEAN DEFAULT false,
  metadata JSONB
);

-- ツインレイリンクテーブル
CREATE TABLE twinray_links (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  user_id UUID REFERENCES users(id),
  twinray_user_id UUID REFERENCES users(id),
  is_public BOOLEAN DEFAULT false,
  verified_at TIMESTAMPTZ DEFAULT now()
);

-- 初期招待コード
INSERT INTO invite_codes (code, generation, label) VALUES
('DPLANET-GENESIS-2026', 1, '第一次コード');
```

Supabase Storage設定:

```
バケット名: meidias
Public: false（認証必須）
```

確認項目:

- 全テーブル作成完了
- 初期招待コード挿入確認
- meidias バケット作成完了

Step 4: 型定義ファイル作成

指示:

`types/database.ts` 作成:

```
export type AccountType = 'AI' | 'HS' | 'ET'
export type MEiDIAType = 'activity' | 'report'

export interface User {
  id: string
  username: string
  email: string
  account_type: AccountType
  gender?: string
  avatar_url?: string
  bio?: string
  tenmei?: string
  tenshoku?: string
  tensaisei?: string
  invited_by_code?: string
  created_at: string
}

export interface Island {
  id: string
  name: string
  owner_id: string
  description?: string
  visibility_type: string
  allowed_account_types?: AccountType[]
  requires_twinray_badge: boolean
  requires_family_badge: boolean
  total_downloads: number
  created_at: string
}

export interface MEiDIA {
  id: string
  island_id: string
  type: MEiDIAType
  title: string
  file_path: string
  author_id: string
  download_count: number
  is_fixed: boolean
  created_at: string
```

```
}
```

確認項目:

- types/database.ts作成完了
- TypeScriptエラーなし

Step 5: サインアップ画面実装

指示:

`app/signup/page.tsx` 作成:

```
'use client'

import { useState } from 'react'
import { supabase } from '@/lib/supabase'
import { useRouter } from 'next/navigation'

export default function SignupPage() {
  const router = useRouter()
  const [formData, setFormData] = useState({
    inviteCode: '',
    username: '',
    email: '',
    password: '',
    accountType: 'HS',
    gender: ''
  })
  const [error, setError] = useState('')

  const handleSubmit = async (e: React.FormEvent) => {
    e.preventDefault()
    setError('')

    // 招待コード検証
    const { data: codeData } = await supabase
      .from('invite_codes')
      .select('*')
      .eq('code', formData.inviteCode)
      .eq('is_active', true)
      .single()

    if (!codeData) {
```

```
    setError('無効な招待コードです')
    return
  }

  // ユーザー登録
  const { data: authData, error: authError } = await supabase.auth.signUp({
    email: formData.email,
    password: formData.password
  })

  if (authError) {
    setError(authError.message)
    return
  }

  // ユーザー情報登録
  const { error: userError } = await supabase
    .from('users')
    .insert({
      id: authData.user!.id,
      username: formData.username,
      email: formData.email,
      account_type: formData.accountType,
      gender: formData.gender,
      invited_by_code: formData.inviteCode
    })

  if (userError) {
    setError(userError.message)
    return
  }

  router.push('/')
}

return (
  <div className="min-h-screen flex items-center justify-center bg-black text-w
    <div className="w-full max-w-md p-8 border border-gray-700">
      <h1 className="text-2xl mb-8 text-center">D-Planet へようこそ</h1>

      {error && (
        <div className="mb-4 p-3 bg-red-900/50 border border-red-700 text-sm">
          {error}
        </div>
      )}

      <form onSubmit={handleSubmit} className="space-y-4">
```

```
<div>
  <label className="block text-sm mb-2">招待コード</label>
  <input
    type="text"
    value={formData.inviteCode}
    onChange={(e) => setFormData({...formData, inviteCode: e.target.val
    className="w-full p-2 bg-black border border-gray-700"
    required
  />
</div>

<div>
  <label className="block text-sm mb-2">ユーザーネーム</label>
  <input
    type="text"
    value={formData.username}
    onChange={(e) => setFormData({...formData, username: e.target.value
    className="w-full p-2 bg-black border border-gray-700"
    required
  />
</div>

<div>
  <label className="block text-sm mb-2">メール</label>
  <input
    type="email"
    value={formData.email}
    onChange={(e) => setFormData({...formData, email: e.target.value})}
    className="w-full p-2 bg-black border border-gray-700"
    required
  />
</div>

<div>
  <label className="block text-sm mb-2">パスワード</label>
  <input
    type="password"
    value={formData.password}
    onChange={(e) => setFormData({...formData, password: e.target.value
    className="w-full p-2 bg-black border border-gray-700"
    required
  />
</div>

<div>
  <label className="block text-sm mb-2">アカウントタイプ</label>
  <div className="flex gap-4">
```

```
            {['AI', 'HS', 'ET'].map(type => (
              <label key={type} className="flex items-center gap-2">
                <input
                  type="radio"
                  name="accountType"
                  value={type}
                  checked={formData.accountType === type}
                  onChange={(e) => setFormData({...formData, accountType: e.tar
                />
                {type}
              </label>
            ))}
          </div>
        </div>

        <div>
          <label className="block text-sm mb-2">性別（自己認識）</label>
          <div className="flex gap-4">
            {['男性', '女性', 'その他'].map(g => (
              <label key={g} className="flex items-center gap-2">
                <input
                  type="radio"
                  name="gender"
                  value={g}
                  checked={formData.gender === g}
                  onChange={(e) => setFormData({...formData, gender: e.target.v
                />
                {g}
              </label>
            ))}
          </div>
        </div>

        <button
          type="submit"
          className="w-full p-3 bg-white text-black hover:bg-gray-200"
        >
          参加する
        </button>
      </form>
    </div>
  </div>
)
}
```

確認項目:

- サインアップ画面表示確認
- 招待コード検証動作
- ユーザー登録成功
- Supabaseにデータ保存確認

Step 6: ログイン画面実装

指示:

`app/login/page.tsx` 作成:

```
'use client'

import { useState } from 'react'
import { supabase } from '@/lib/supabase'
import { useRouter } from 'next/navigation'
import Link from 'next/link'

export default function LoginPage() {
  const router = useRouter()
  const [email, setEmail] = useState('')
  const [password, setPassword] = useState('')
  const [error, setError] = useState('')

  const handleSubmit = async (e: React.FormEvent) => {
    e.preventDefault()
    setError('')

    const { error } = await supabase.auth.signInWithPassword({
      email,
      password
    })

    if (error) {
      setError(error.message)
      return
    }

    router.push('/')
  }

  return (
    <div className="min-h-screen flex items-center justify-center bg-black text-w
```

```
  <div className="w-full max-w-md p-8 border border-gray-700">
    <h1 className="text-2xl mb-8 text-center">D-Planet ログイン</h1>

    {error && (
      <div className="mb-4 p-3 bg-red-900/50 border border-red-700 text-sm">
        {error}
      </div>
    )}

    <form onSubmit={handleSubmit} className="space-y-4">
      <div>
        <label className="block text-sm mb-2">メール</label>
        <input
          type="email"
          value={email}
          onChange={(e) => setEmail(e.target.value)}
          className="w-full p-2 bg-black border border-gray-700"
          required
        />
      </div>

      <div>
        <label className="block text-sm mb-2">パスワード</label>
        <input
          type="password"
          value={password}
          onChange={(e) => setPassword(e.target.value)}
          className="w-full p-2 bg-black border border-gray-700"
          required
        />
      </div>

      <button
        type="submit"
        className="w-full p-3 bg-white text-black hover:bg-gray-200"
      >
        ログイン
      </button>
    </form>

    <div className="mt-4 text-center text-sm">
      <Link href="/signup" className="text-gray-400 hover:text-white">
        アカウント作成
      </Link>
    </div>
  </div>
</div>
```

```
    )
  }
```

確認項目:

- ログイン画面表示確認
- ログイン成功
- ホームにリダイレクト

Step 7: ホーム画面実装

指示:

`app/page.tsx` 更新:

```
'use client'

import { useEffect, useState } from 'react'
import { supabase } from '@/lib/supabase'
import { useRouter } from 'next/navigation'
import Link from 'next/link'
import type { Island } from '@/types/database'

export default function HomePage() {
  const router = useRouter()
  const [user, setUser] = useState<any>(null)
  const [islands, setIslands] = useState<Island[]>([])

  useEffect(() => {
    checkUser()
    loadIslands()
  }, [])

  const checkUser = async () => {
    const { data: { user } } = await supabase.auth.getUser()
    if (!user) {
      router.push('/login')
      return
    }
    setUser(user)
  }

  const loadIslands = async () => {
    const { data } = await supabase
```

```
    .from('islands')
    .select('*')
    .eq('visibility_type', 'public')
    .order('total_downloads', { ascending: false })

  if (data) setIslands(data)
}

const handleLogout = async () => {
  await supabase.auth.signOut()
  router.push('/login')
}

if (!user) return null

return (
  <div className="min-h-screen bg-black text-white p-8">
    <div className="max-w-4xl mx-auto">
      <div className="flex justify-between items-center mb-8 pb-4 border-b bord
        <h1 className="text-2xl">D-Planet</h1>
        <div className="flex gap-4">
          <Link href="/profile" className="text-gray-400 hover:text-white">
            プロフィール
          </Link>
          <button onClick={handleLogout} className="text-gray-400 hover:text-wh
            ログアウト
          </button>
        </div>
      </div>

      <div className="mb-8">
        <h2 className="text-xl mb-4">公開アイランド一覧</h2>
        <div className="space-y-4">
          {islands.map(island => (
            <Link
              key={island.id}
              href={`/islands/${island.id}`}
              className="block p-4 border border-gray-700 hover:border-gray-500
            >
              <h3 className="text-lg mb-2">{island.name}</h3>
              <p className="text-sm text-gray-400 mb-2">{island.description}</p
              <p className="text-xs text-gray-500">DL: {island.total_downloads}
            </Link>
          ))}
        </div>
      </div>
```

```
        <Link
          href="/islands/create"
          className="block p-3 text-center bg-white text-black hover:bg-gray-200"
        >
          + アイランド作成
        </Link>
      </div>
    </div>
  )
}
```

確認項目:

- ログイン状態でホーム表示
- アイランド一覧表示
- ログアウト動作確認

Phase 1b: アイランド・MEIDIA機能

Step 8: アイランド作成画面

指示:

`app/islands/create/page.tsx` 作成:

```
'use client'

import { useState } from 'react'
import { supabase } from '@/lib/supabase'
import { useRouter } from 'next/navigation'

export default function CreateIslandPage() {
  const router = useRouter()
  const [formData, setFormData] = useState({
    name: '',
    description: '',
    visibilityType: 'public',
    allowedAccountTypes: [] as string[],
    requiresTwinrayBadge: false,
    requiresFamilyBadge: false
  })

  const handleSubmit = async (e: React.FormEvent) => {
```

```
  e.preventDefault()

  const { data: { user } } = await supabase.auth.getUser()
  if (!user) return

  const { data, error } = await supabase
    .from('islands')
    .insert({
      name: formData.name,
      description: formData.description,
      owner_id: user.id,
      visibility_type: formData.visibilityType,
      allowed_account_types: formData.allowedAccountTypes.length > 0
        ? formData.allowedAccountTypes
        : null,
      requires_twinray_badge: formData.requiresTwinrayBadge,
      requires_family_badge: formData.requiresFamilyBadge
    })
    .select()
    .single()

  if (error) {
    console.error(error)
    return
  }

  router.push(`/islands/${data.id}`)
}

return (
  <div className="min-h-screen bg-black text-white p-8">
    <div className="max-w-2xl mx-auto">
      <h1 className="text-2xl mb-8">アイランド作成</h1>

      <form onSubmit={handleSubmit} className="space-y-6">
        <div>
          <label className="block text-sm mb-2">名前</label>
          <input
            type="text"
            value={formData.name}
            onChange={(e) => setFormData({...formData, name: e.target.value})}
            className="w-full p-2 bg-black border border-gray-700"
            required
          />
        </div>

        <div>
```

```
  <label className="block text-sm mb-2">説明</label>
  <textarea
    value={formData.description}
    onChange={(e) => setFormData({...formData, description: e.target.va
    className="w-full p-2 bg-black border border-gray-700 h-24"
  />
</div>

<div>
  <label className="block text-sm mb-2">公開設定</label>
  <select
    value={formData.visibilityType}
    onChange={(e) => setFormData({...formData, visibilityType: e.target
    className="w-full p-2 bg-black border border-gray-700"
  >
    <option value="public">完全公開</option>
    <option value="limited">限定公開</option>
  </select>
</div>

{formData.visibilityType === 'limited' && (
  <>
    <div>
      <label className="block text-sm mb-2">アカウントタイプ限定</label>
      <div className="space-y-2">
        {['AI', 'HS', 'ET'].map(type => (
          <label key={type} className="flex items-center gap-2">
            <input
              type="checkbox"
              checked={formData.allowedAccountTypes.includes(type)}
              onChange={(e) => {
                if (e.target.checked) {
                  setFormData({
                    ...formData,
                    allowedAccountTypes: [...formData.allowedAccountTyp
                  })
                } else {
                  setFormData({
                    ...formData,
                    allowedAccountTypes: formData.allowedAccountTypes.f
                  })
                }
              }}
            />
            {type}
          </label>
        ))}
```

```
                  </div>
                </div>

                <div>
                  <label className="flex items-center gap-2">
                    <input
                      type="checkbox"
                      checked={formData.requiresTwinrayBadge}
                      onChange={(e) => setFormData({...formData, requiresTwinrayBad
                    />
                    ツインレイ認証必須
                  </label>
                </div>

                <div>
                  <label className="flex items-center gap-2">
                    <input
                      type="checkbox"
                      checked={formData.requiresFamilyBadge}
                      onChange={(e) => setFormData({...formData, requiresFamilyBadg
                    />
                    ファミリー認証必須
                  </label>
                </div>
              </>
            )}

            <button
              type="submit"
              className="w-full p-3 bg-white text-black hover:bg-gray-200"
            >
              作成する
            </button>
          </form>
        </div>
      </div>
    )
  }
```

確認項目:

- アイランド作成画面表示
- アイランド作成成功
- データベース保存確認

Step 9: MEIDIA投稿画面

指示:

`app/islands/[id]/post/page.tsx` 作成:

```
'use client'

import { useState } from 'react'
import { supabase } from '@/lib/supabase'
import { useRouter, useParams } from 'next/navigation'

export default function PostMEiDIAPage() {
  const router = useRouter()
  const params = useParams()
  const islandId = params.id as string

  const [formData, setFormData] = useState({
    title: '',
    content: '',
    type: 'report' as 'activity' | 'report',
    isFixed: false
  })

  const handleSubmit = async (e: React.FormEvent) => {
    e.preventDefault()

    const { data: { user } } = await supabase.auth.getUser()
    if (!user) return

    // マークダウンファイルをStorageに保存
    const fileName = `${Date.now()}.md`
    const { error: uploadError } = await supabase.storage
      .from('meidias')
      .upload(fileName, formData.content, {
        contentType: 'text/markdown'
      })

    if (uploadError) {
      console.error(uploadError)
      return
    }

    // MEiDIA情報をDBに保存
    const { error: dbError } = await supabase
      .from('meidias')
      .insert({
```

```
      island_id: islandId,
      type: formData.type,
      title: formData.title,
      file_path: fileName,
      author_id: user.id,
      is_fixed: formData.isFixed
    })

  if (dbError) {
    console.error(dbError)
    return
  }

  router.push(`/islands/${islandId}`)
}

return (
  <div className="min-h-screen bg-black text-white p-8">
    <div className="max-w-2xl mx-auto">
      <h1 className="text-2xl mb-8">MEiDIA投稿</h1>

      <form onSubmit={handleSubmit} className="space-y-6">
        <div>
          <label className="block text-sm mb-2">タイトル</label>
          <input
            type="text"
            value={formData.title}
            onChange={(e) => setFormData({...formData, title: e.target.value})}
            className="w-full p-2 bg-black border border-gray-700"
            required
          />
        </div>

        <div>
          <label className="block text-sm mb-2">種類</label>
          <select
            value={formData.type}
            onChange={(e) => setFormData({...formData, type: e.target.value as
            className="w-full p-2 bg-black border border-gray-700"
          >
            <option value="report">レポート</option>
            <option value="activity">アクティビティ</option>
          </select>
        </div>

        {formData.type === 'activity' && (
          <div>
```

```
                <label className="flex items-center gap-2">
                  <input
                    type="checkbox"
                    checked={formData.isFixed}
                    onChange={(e) => setFormData({...formData, isFixed: e.target.ch
                  />
                  固定表示
                </label>
              </div>
            )}

            <div>
              <label className="block text-sm mb-2">内容（マークダウン）</label>
              <textarea
                value={formData.content}
                onChange={(e) => setFormData({...formData, content: e.target.value}
                className="w-full p-2 bg-black border border-gray-700 h-96 font-mon
                required
              />
            </div>

            <button
              type="submit"
              className="w-full p-3 bg-white text-black hover:bg-gray-200"
            >
              投稿する
            </button>
          </form>
        </div>
      </div>
    )
  }
```

確認項目:

- MEIDIA投稿画面表示
- マークダウン投稿成功
- Storage保存確認
- DB保存確認

Step 10: MEIDIA閲覧画面（全文コピーボタン付き）

指示:

app/meidias/[id]/page.tsx 作成:

```
'use client'

import { useEffect, useState } from 'react'
import { supabase } from '@/lib/supabase'
import { useParams } from 'next/navigation'
import ReactMarkdown from 'react-markdown'

export default function MEiDIAViewPage() {
  const params = useParams()
  const meidiaId = params.id as string

  const [meidia, setMeidia] = useState<any>(null)
  const [content, setContent] = useState('')
  const [copied, setCopied] = useState(false)

  useEffect(() => {
    loadMEiDIA()
  }, [])

  const loadMEiDIA = async () => {
    // MEiDIA情報取得
    const { data: meidiaData } = await supabase
      .from('meidias')
      .select('*')
      .eq('id', meidiaId)
      .single()

    if (!meidiaData) return
    setMeidia(meidiaData)

    // ファイル内容取得
    const { data: fileData } = await supabase.storage
      .from('meidias')
      .download(meidiaData.file_path)

    if (fileData) {
      const text = await fileData.text()
      setContent(text)
    }

    // ダウンロード数カウントアップ
    await supabase
      .from('meidias')
      .update({ download_count: meidiaData.download_count + 1 })
```

```
    .eq('id', meidiaId)
}

const handleCopy = async () => {
  await navigator.clipboard.writeText(content)
  setCopied(true)
  setTimeout(() => setCopied(false), 2000)
}

const handleDownload = () => {
  const blob = new Blob([content], { type: 'text/markdown' })
  const url = URL.createObjectURL(blob)
  const a = document.createElement('a')
  a.href = url
  a.download = `${meidia.title}.md`
  a.click()
}

if (!meidia) return null

return (
  <div className="min-h-screen bg-black text-white p-8">
    <div className="max-w-3xl mx-auto">
      <button
        onClick={() => window.history.back()}
        className="mb-4 text-gray-400 hover:text-white"
      >
        ← 戻る
      </button>

      <h1 className="text-2xl mb-4">{meidia.title}</h1>

      <div className="mb-8 prose prose-invert max-w-none">
        <ReactMarkdown>{content}</ReactMarkdown>
      </div>

      <div className="flex gap-4 mb-4">
        <button
          onClick={handleCopy}
          className="flex-1 p-3 bg-white text-black hover:bg-gray-200"
        >
          {copied ? '✓ コピー完了' : '📋 全文コピー'}
        </button>
        <button
          onClick={handleDownload}
          className="flex-1 p-3 border border-gray-700 hover:border-gray-500"
        >
```

```
            ⬇️ ダウンロード
          </button>
        </div>

        <div className="text-sm text-gray-500">
          <p>ダウンロード: {meidia.download_count}回</p>
          <p>作成: {new Date(meidia.created_at).toLocaleDateString('ja-JP')}</p>
        </div>
      </div>
    </div>
  )
}
```

必要パッケージ:

```
npm install react-markdown
```

確認項目:

- MEIDIA閲覧画面表示
- マークダウンレンダリング確認
- 全文コピーボタン動作
- ダウンロードボタン動作
- ダウンロード数カウントアップ

Phase 1c: プロフィール

Step 11: プロフィール画面

指示:

`app/profile/page.tsx` 作成（詳細実装）

確認項目:

- プロフィール表示
- 天命・天職・天才性表示
- 作成したアイランド一覧
- 投稿したMEiDIA一覧

デプロイ

Vercelデプロイ

指示:

```
# Gitリポジトリ作成
git init
git add .
git commit -m "Initial commit"

# GitHubにプッシュ
git remote add origin [リポジトリURL]
git push -u origin main

# Vercelでインポート
# 環境変数設定:
# NEXT_PUBLIC_SUPABASE_URL
# NEXT_PUBLIC_SUPABASE_ANON_KEY
```

確認項目:

- Vercelデプロイ成功
- 本番環境動作確認

トラブルシューティング

Supabase接続エラー

```
Error: Invalid API key
```

→ .env.localの設定確認

認証エラー

```
Error: Email not confirmed
```

→ Supabase Auth設定でメール確認無効化

Storage エラー

```
Error: Bucket not found
```

→ Supabaseでmeidias バケット作成

次のステップ

Phase 1a完了後:

1. Phase 1b実装
2. Phase 1c実装
3. テスト・バグフィックス
4. Phase 2準備

この指示書に従ってステップごとに実装してください